# Deep Learning and its Applications to Pollen Analysis

Lecture 5: Segmentation and Detection

**Shu Kong**

University of Macau
https://aimerykong.github.io/
skong@um.edu.mo

43rd Mid-Continent Paleobotanical Colloquium
Panama 2026

# Roadmap

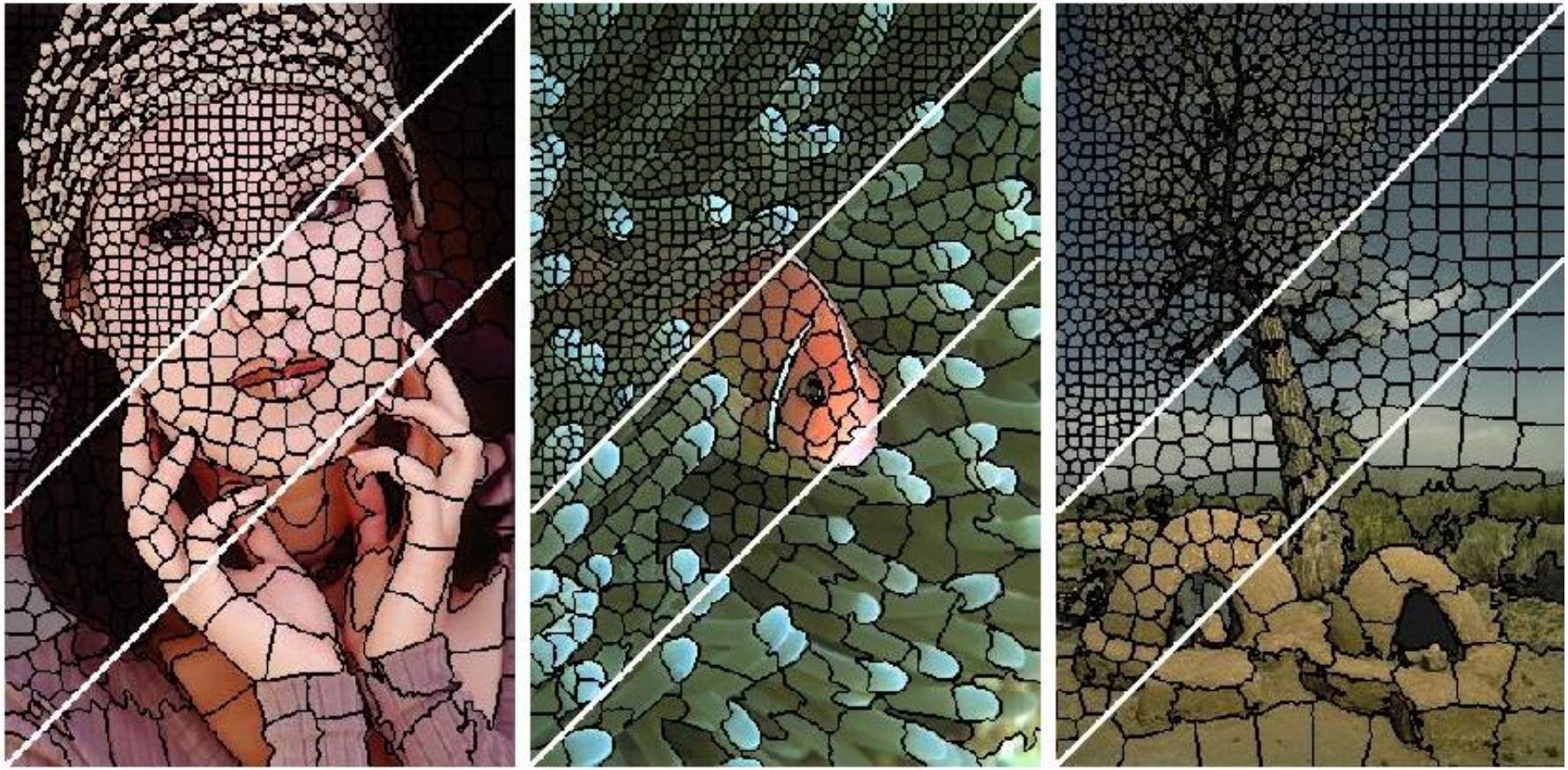
**Toolboxes**:

- platform: Python, Jupyter Notebook
- basics: sklearn, skimage, numpy, scipy, matplotlib
- for deep learning: PyTorch

| Lecture | Topics |
|---|---|
| 1. Introduction | machine learning, computer vision, deep learning, artificial intelligence |
| 2. Basics via Classification | cross entropy, loss function, optimization, feature, learning rate, gradient descent, regularization, etc. |
| 3. Deep Neural Networks | Neural Network, pre-processing, data augmentation, feature learning, pretraining, finetuning |
| 4. Hands-On Practice: Basics | pollen data preparation, visualization, nearest neighbor, linear classifier, pretrained / foundation model |
| 5. Segmentation and Detection | semantic segmentation, instance segmentation, U-Net, proposal detection, foreground segmentation |
| 6. Foundation Models and Finetuning | CLIP, Segment Anything Model, DINOv2, finetuning, model adaptation |
| 7. Advanced Topics | novel species recognition, phylogentic regression, phylogentic reconstruction via clustering |
| 8. Hands-On Practice: Advanced Topics | pollen detection, finetuning foundation models |

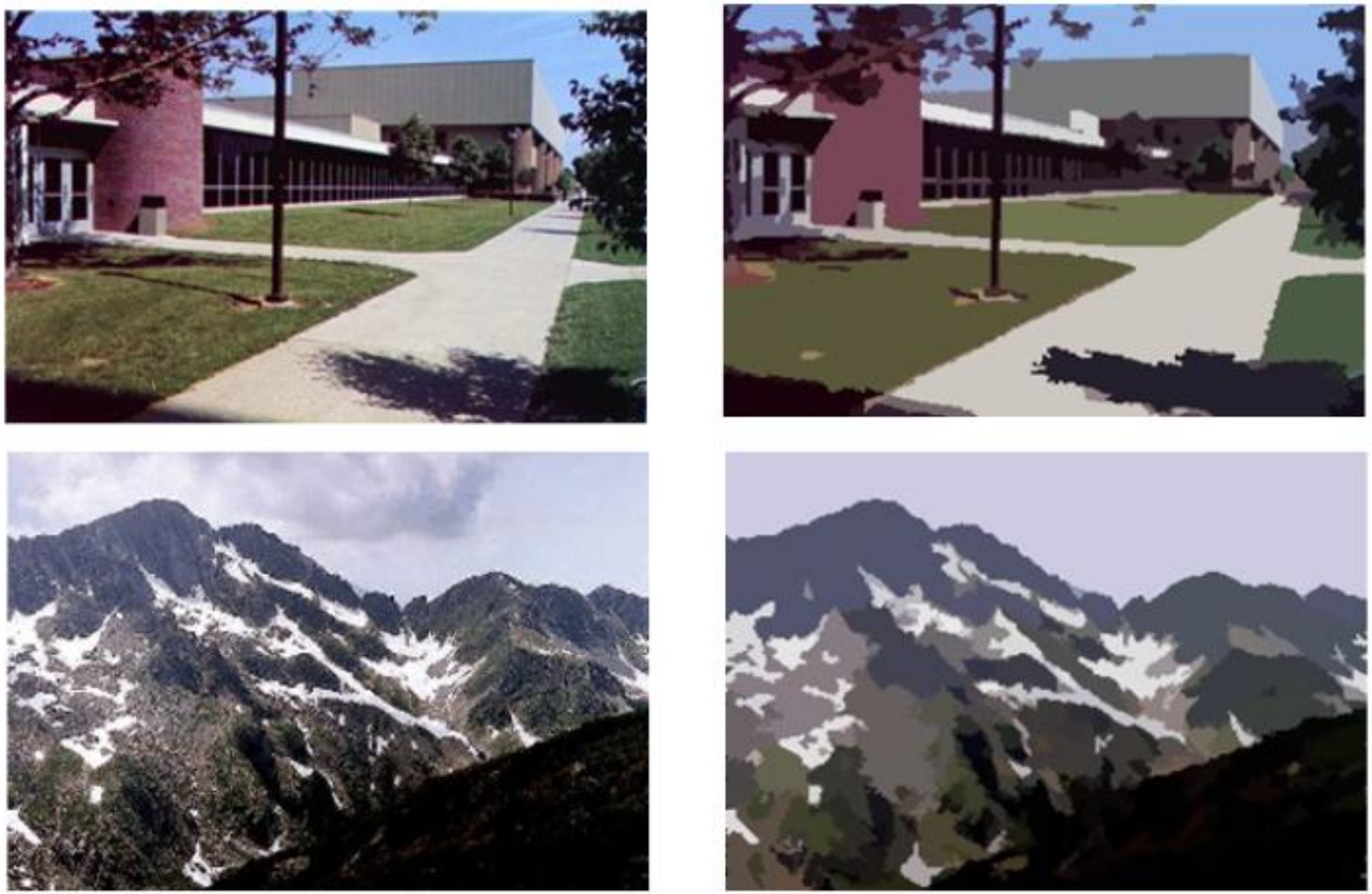
# The goals of segmentation

- Group together similar-looking pixels for efficiency of further processing



# The goals of segmentation

- Group together similar-looking pixels for efficiency of further processing



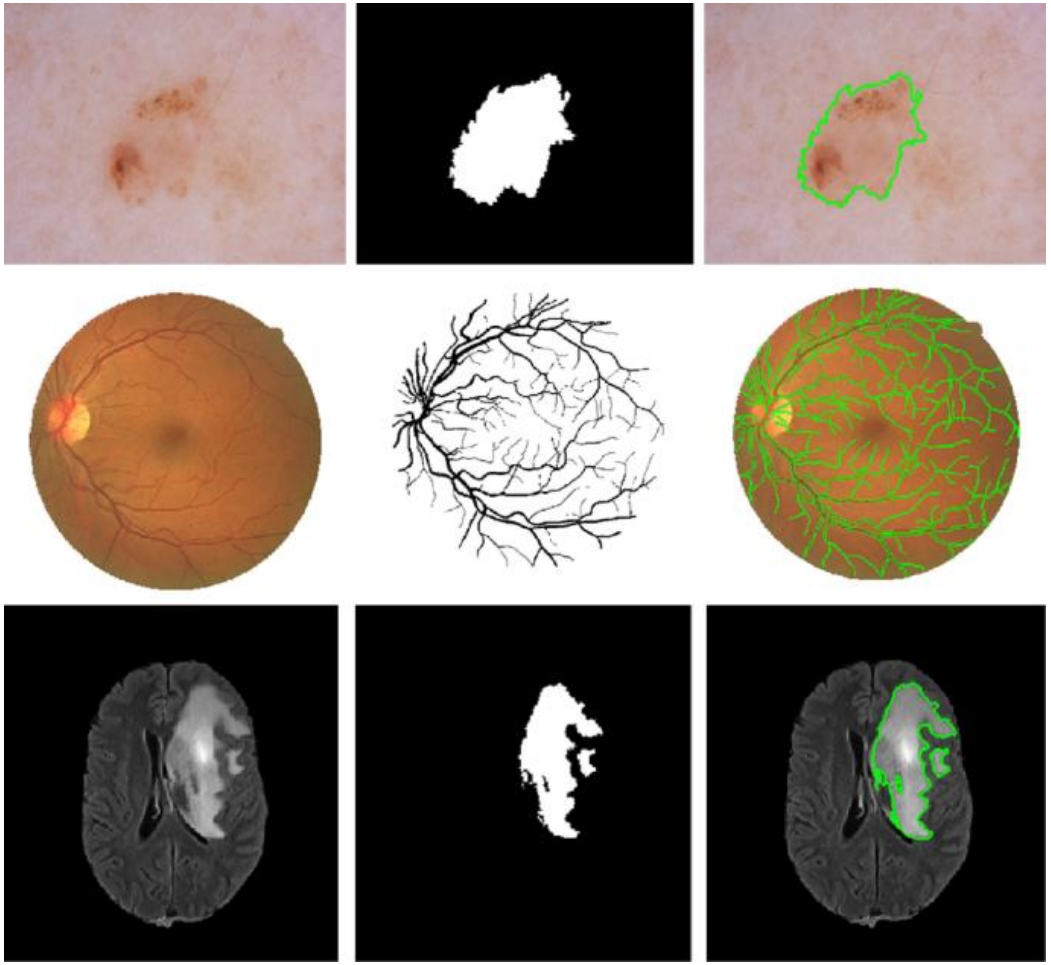
# The goals of segmentation

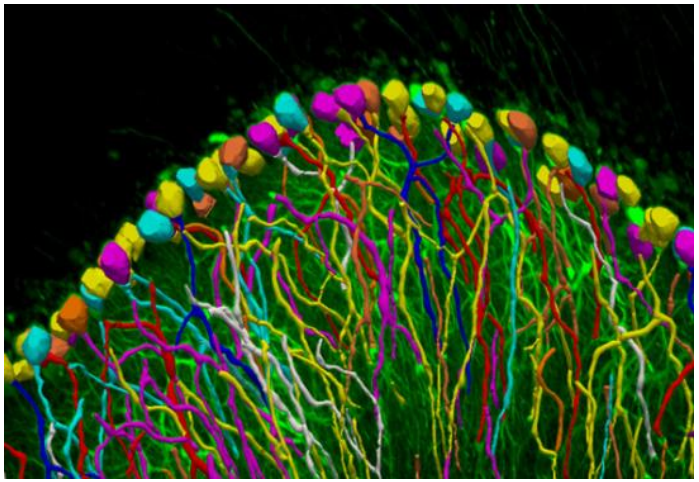
- Group together similar-looking pixels for efficiency of further processing

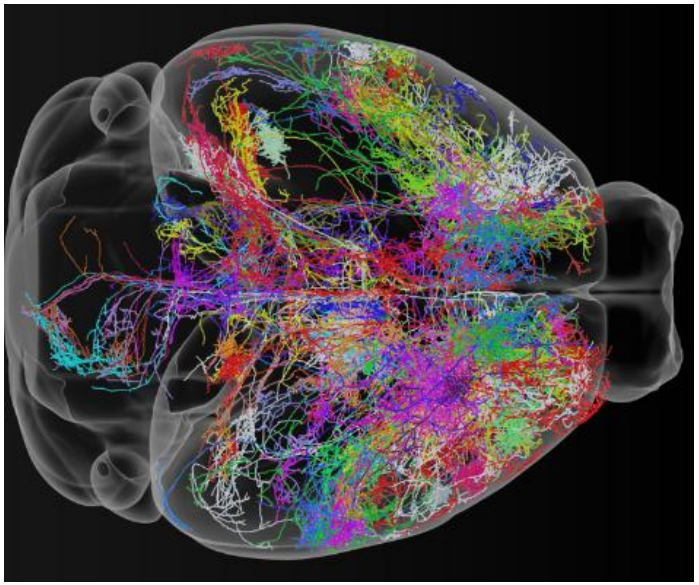
Image compression

# The goals of segmentation

- Group together similar-looking pixels for efficiency of further processing
- Separate image into coherent “objects”

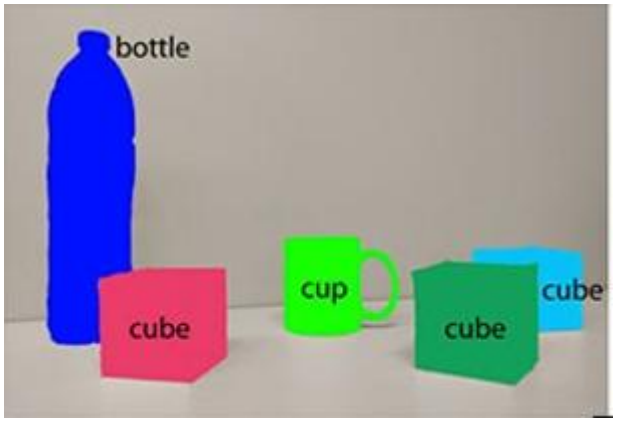






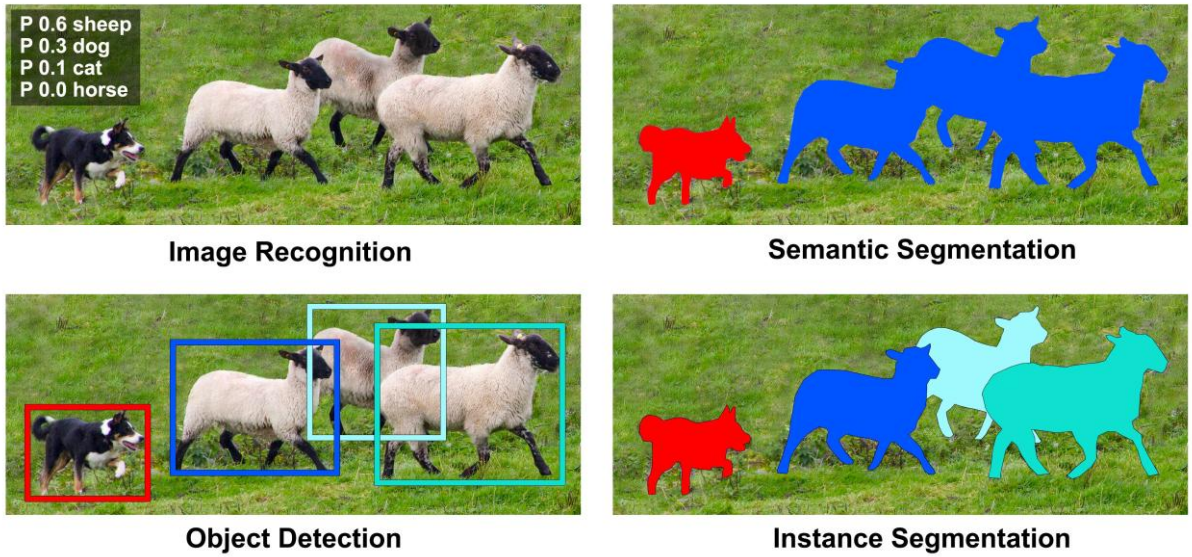
# The goals of segmentation

- Group together similar-looking pixels for efficiency of further processing
- Separate image into coherent “objects”



# The goals of segmentation

- Group together similar-looking pixels for efficiency of further processing
- Separate image into coherent “objects”



# The goals of segmentation

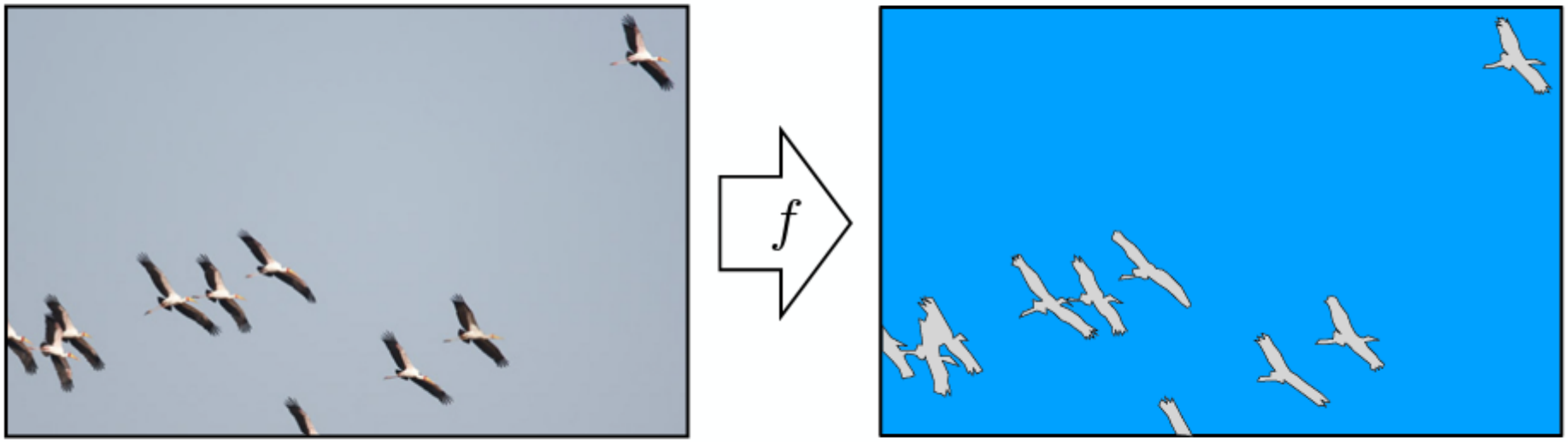
- Group together similar-looking pixels for efficiency of further processing
- Separate image into coherent “objects”

# Outline

- **Deep Learning for Semantic Segmentation**
- Deep Learning for Instance Segmentation

Deep Learning for Semantic Segmentation

# Semantic segmentation




(Colors represent categories)



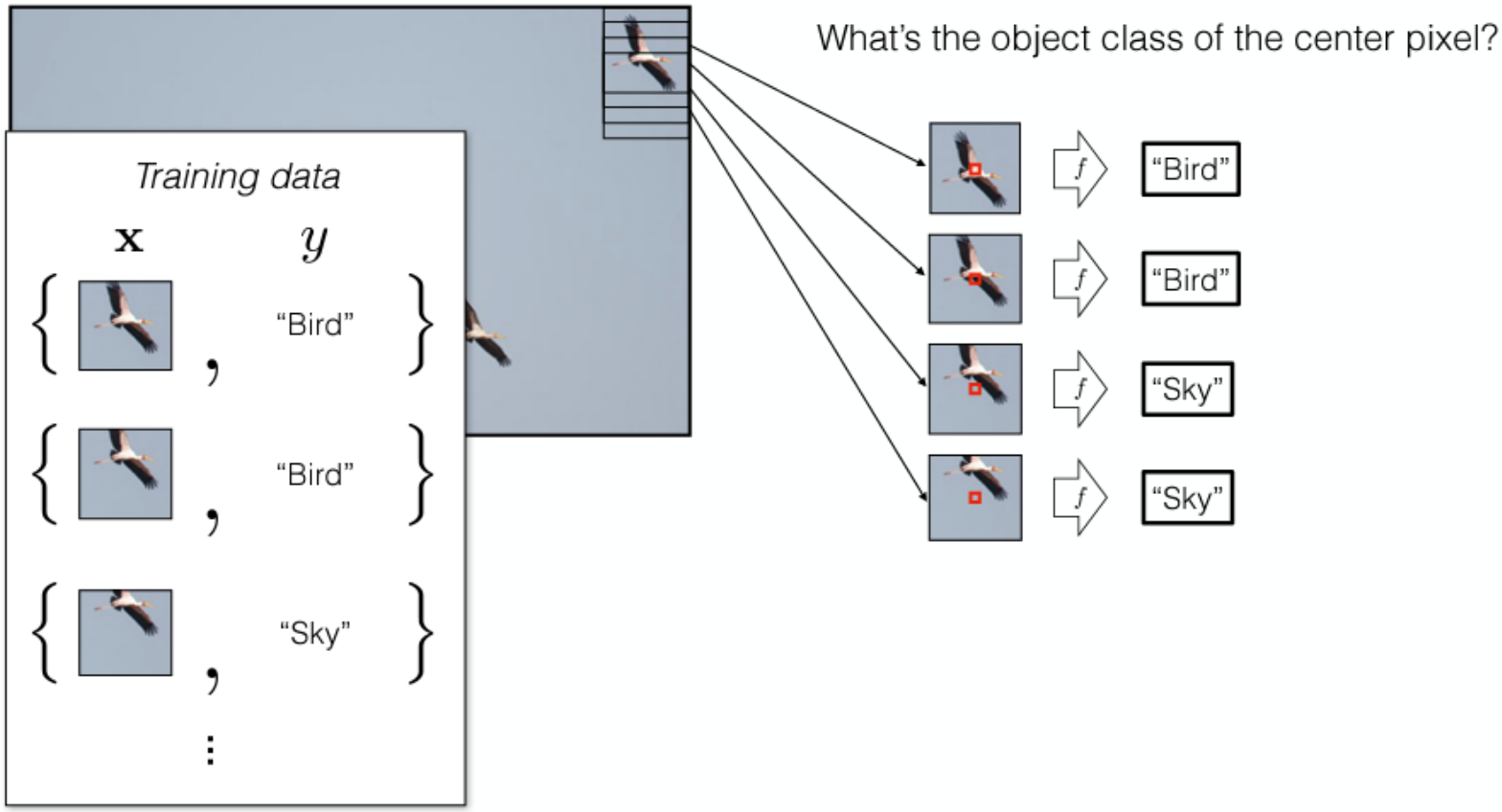
**General technique:** predict something at every pixel!

# Deep Learning for Semantic Segmentation

- Idea 1: independently classify windows

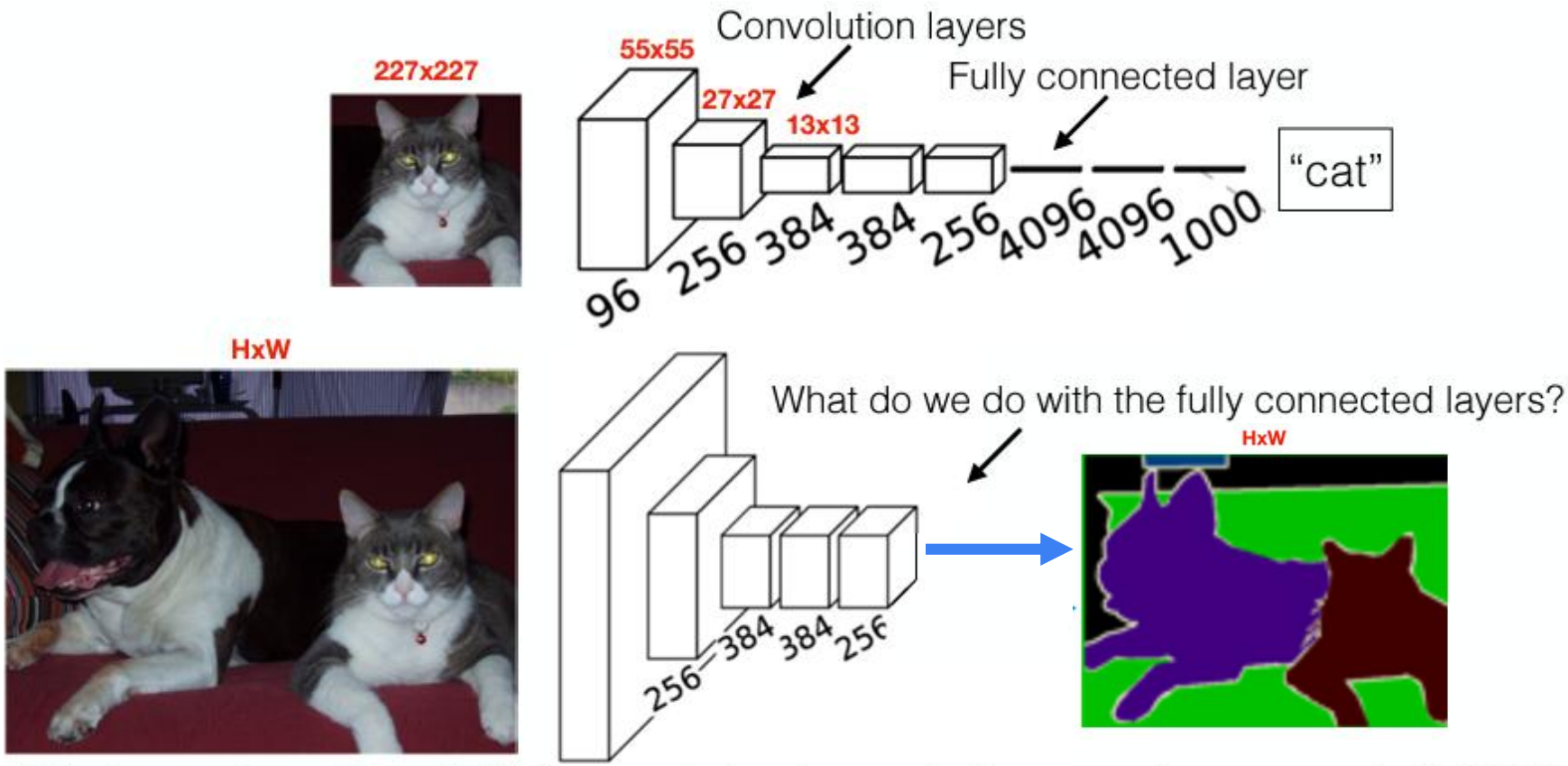
# Deep Learning for Semantic Segmentation

- Idea 1: independently classify windows



# Deep Learning for Semantic Segmentation

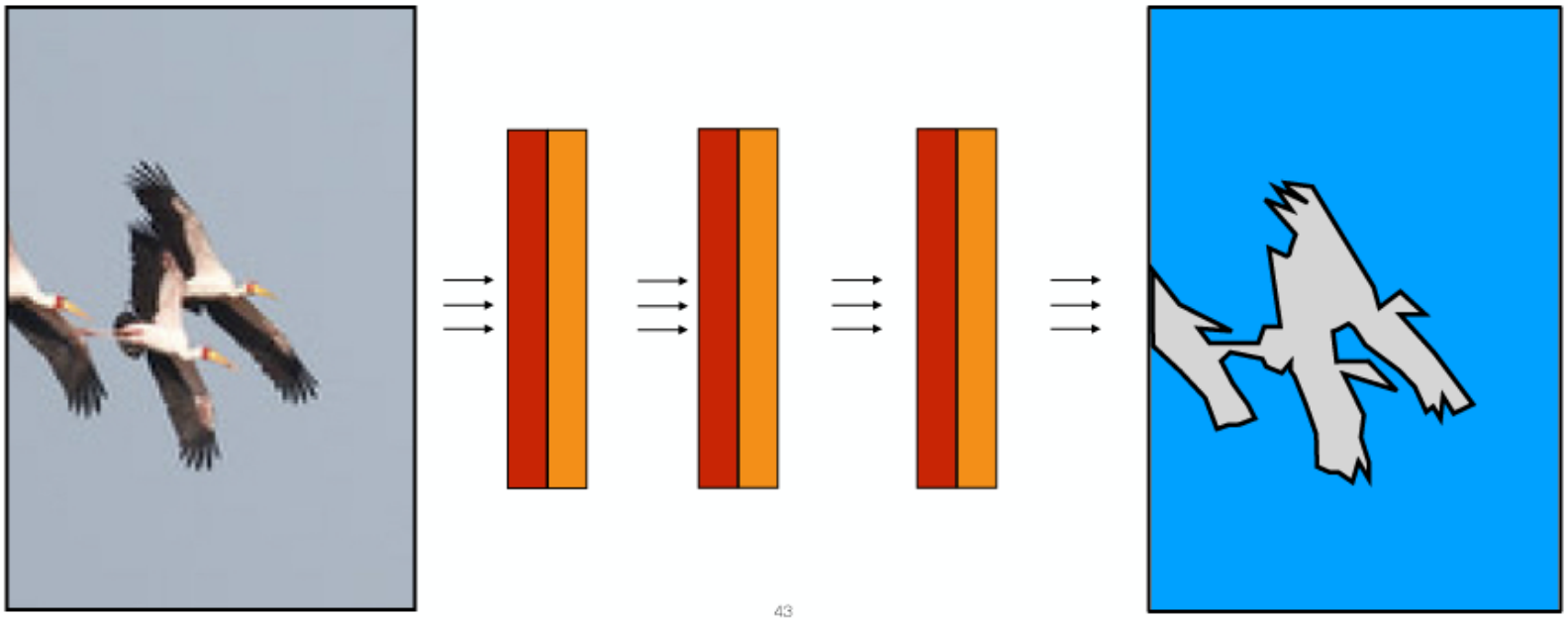
- Idea 1: independently classify windows
- Idea 2: convolutionally classify pixels




[Shelhamer, Long, Darrell, "Fully convolutional networks for semantic segmentation", CVPR 2015]

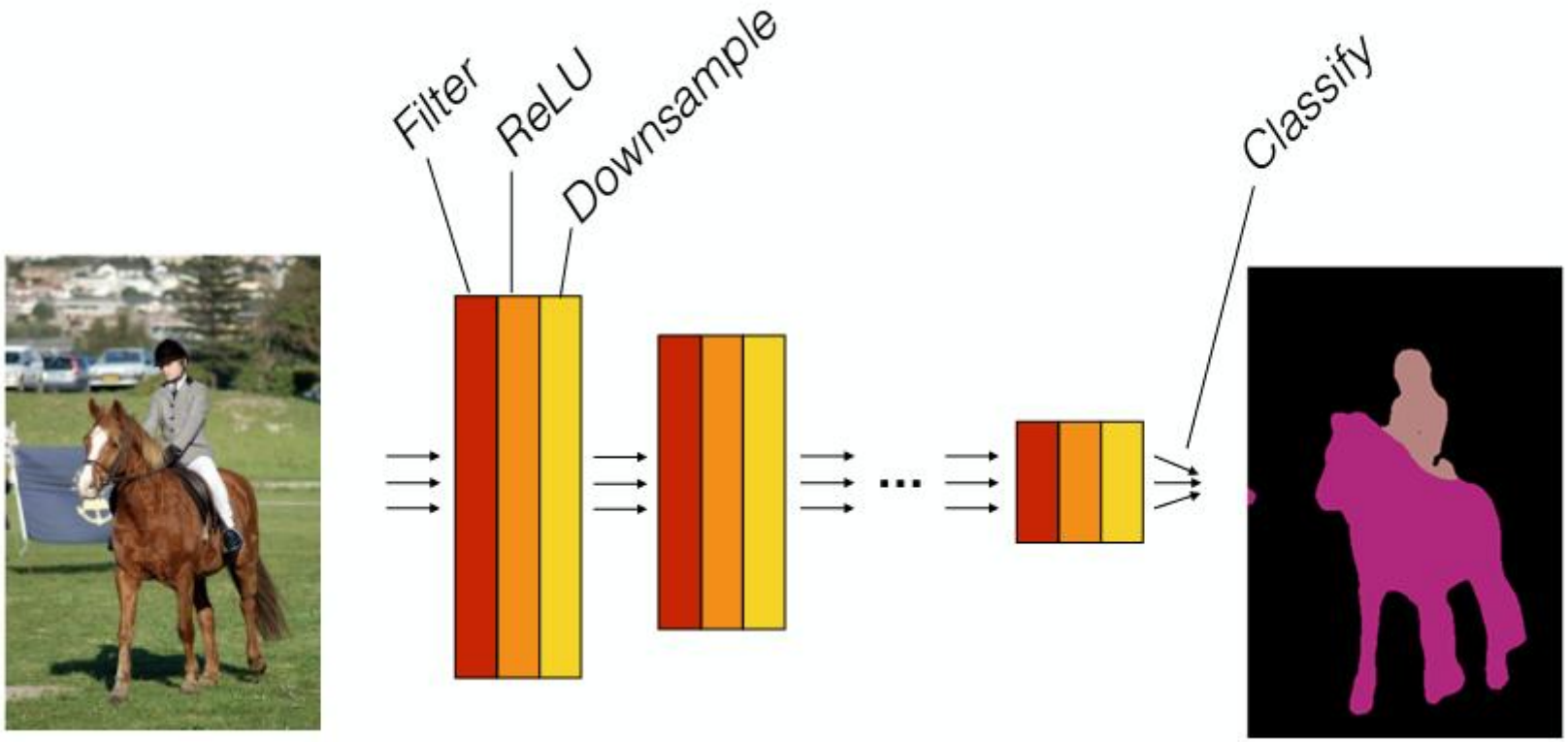
# Deep Learning for Semantic Segmentation

What if we remove subsampling layers?




- Small receptive fields
- Computationally expensive

# Deep Learning for Semantic Segmentation



# Upsampling

| | |
|---|---|
| 1 | 2 |
| 3 | 4 |

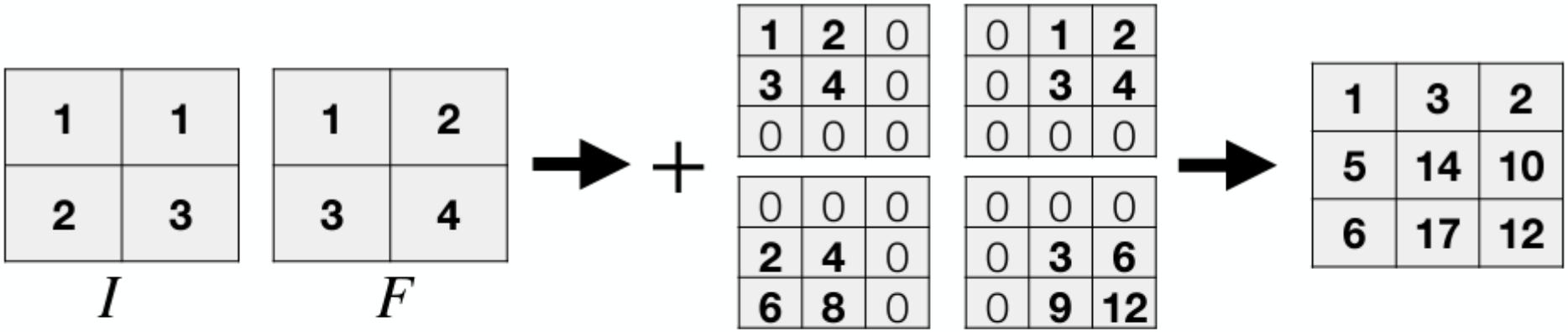
→

| | | | |
|---|---|---|---|
| 1 | 1 | 2 | 2 |
| 1 | 1 | 2 | 2 |
| 3 | 3 | 4 | 4 |
| 3 | 3 | 4 | 4 |

→ Convolution

- Often using nearest-neighbor upsampling
- Can also use interpolation.
- Produces fewer “checkerboard” artifacts

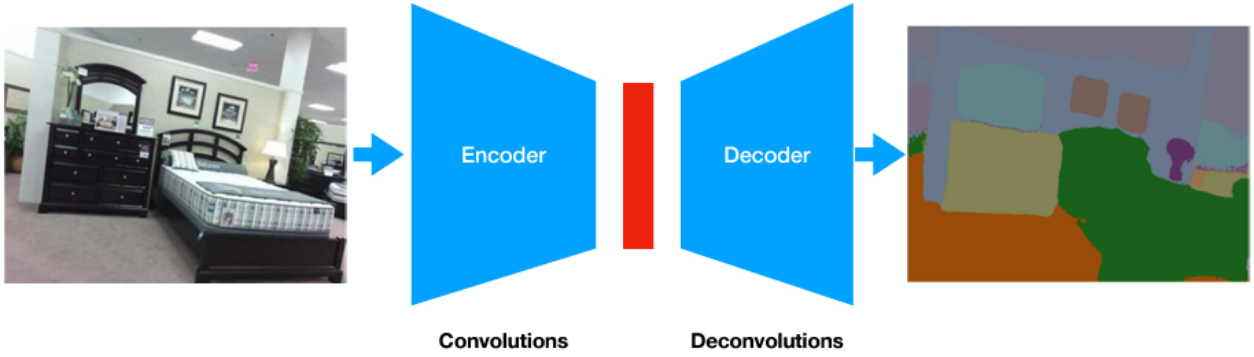
# Transposed convolution




- Weight the filter by the image coefficient and sum.
- Also sometimes called “upconvolution” or “deconvolution”.

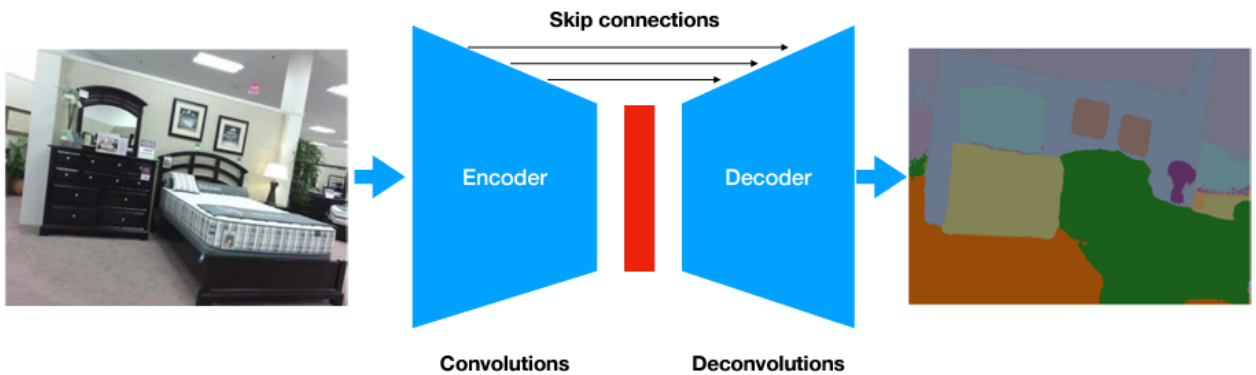
Deep Learning for Semantic Segmentation

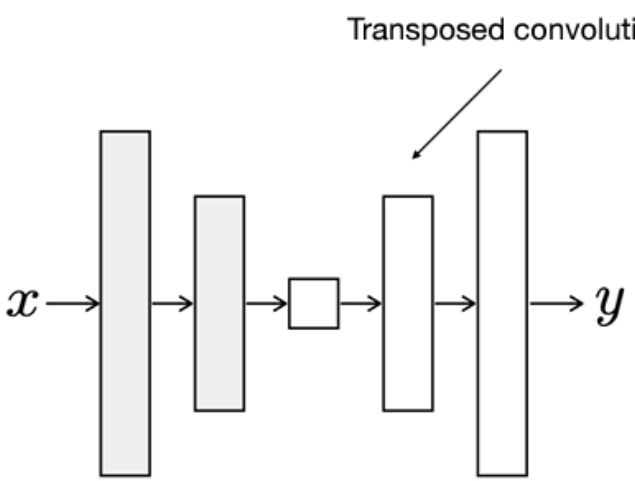
# Encoder-decoder architectures



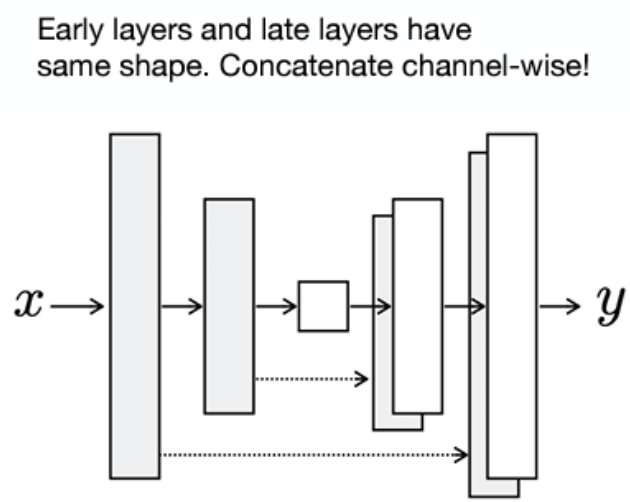
# Deep Learning for Semantic Segmentation

## Encoder-decoder architectures



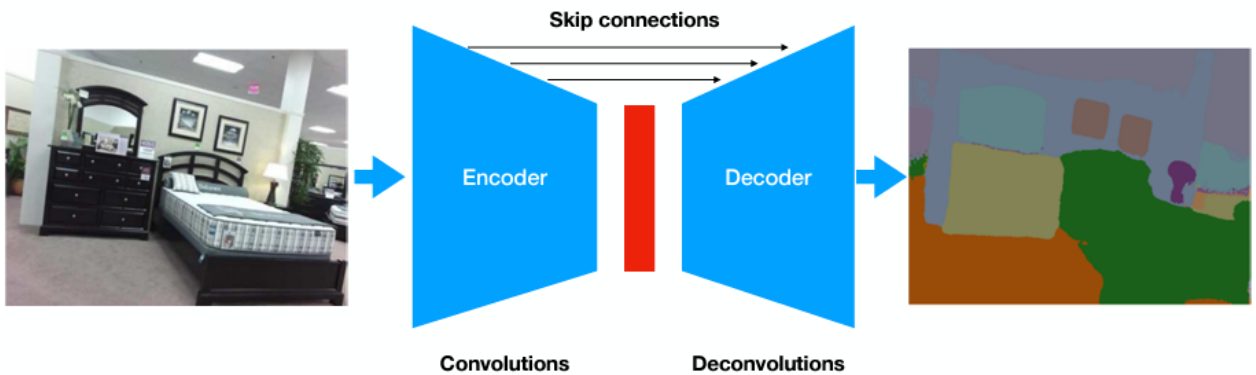

"Vanilla" encoder-decoder architecture




U-Net

# Deep Learning for Semantic Segmentation

## Encoder-decoder architectures




- This is essentially multi-way classification over all pixels
- Apply cross-entropy loss over all pixels

$$J(\theta) = -\frac{1}{m}\sum_i \log p(y^{(i)}|x^{(i)}) + \alpha\,\theta\theta^T$$

$$p(Y = y|X = x) = \frac{\exp(\theta_y \cdot x^T)}{\sum_c \exp(\theta_c \cdot x^T)}$$

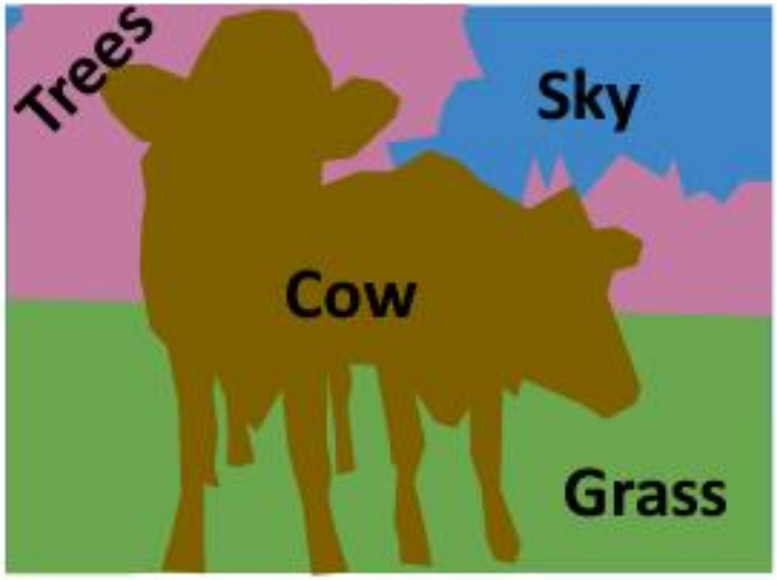
# Outline

- Deep Learning for Semantic Segmentation
- **Deep Learning for Instance Segmentation**

## Things and Stuff

**Things**: Object categories that can be separated into object instances
(e.g. cats, cars, person)

**Stuff**: Object categories that cannot be separated into instances
(e.g. sky, grass, water, trees)



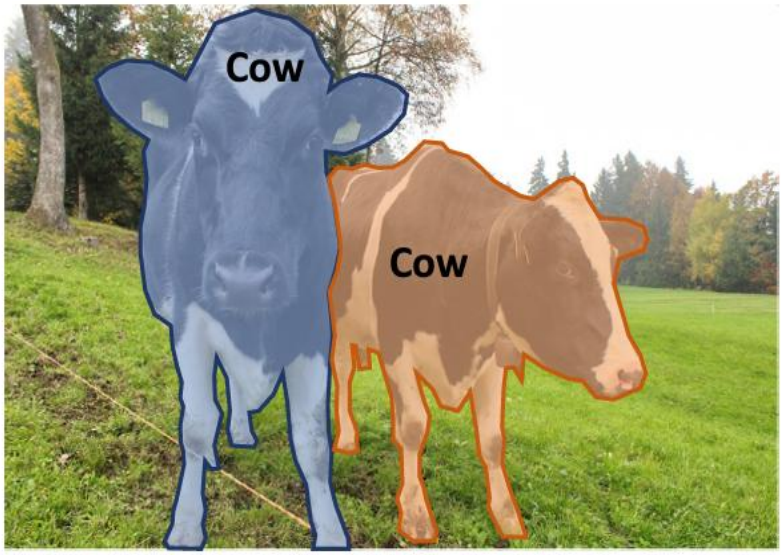
# Deep Learning for Instance Segmentation

## Things and Stuff

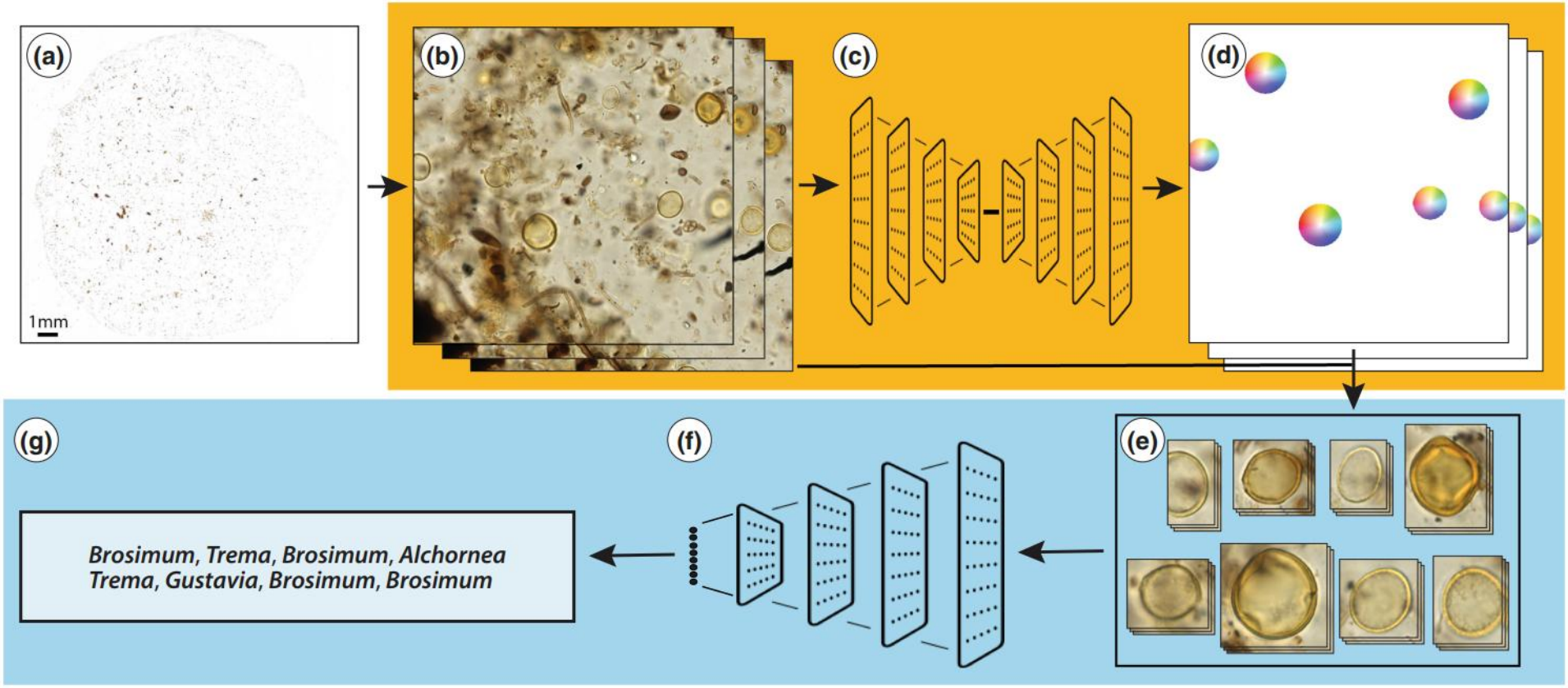
**Things**: Object categories that can be separated into object instances
(e.g. cats, cars, person)

**Stuff**: Object categories that cannot be separated into instances
(e.g. sky, grass, water, trees)

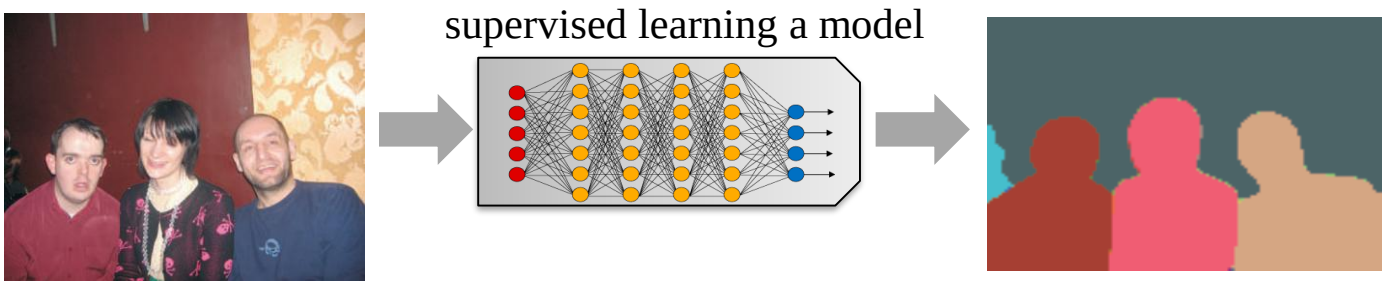
**Instance Segmentation**: Detect all objects in the image, and identify the pixels that belong to each object (Only things!)



# Deep Learning for Pollen Grain Segmentation/Detection

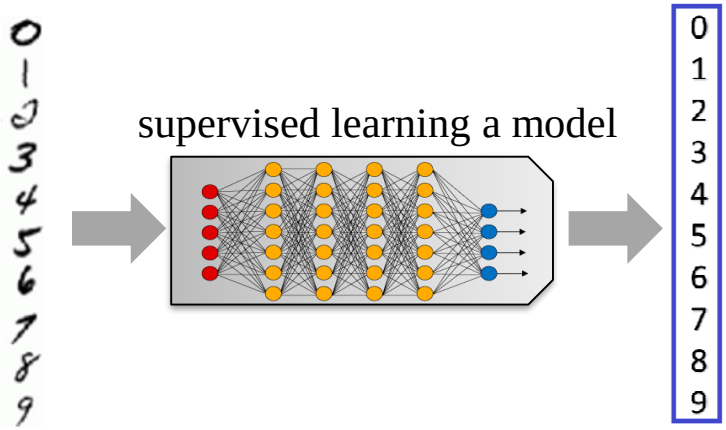


# Instance Segmentation




Alexe et al., "What is an object?" CVPR, 2010

# Recall Deep Learning for Classification

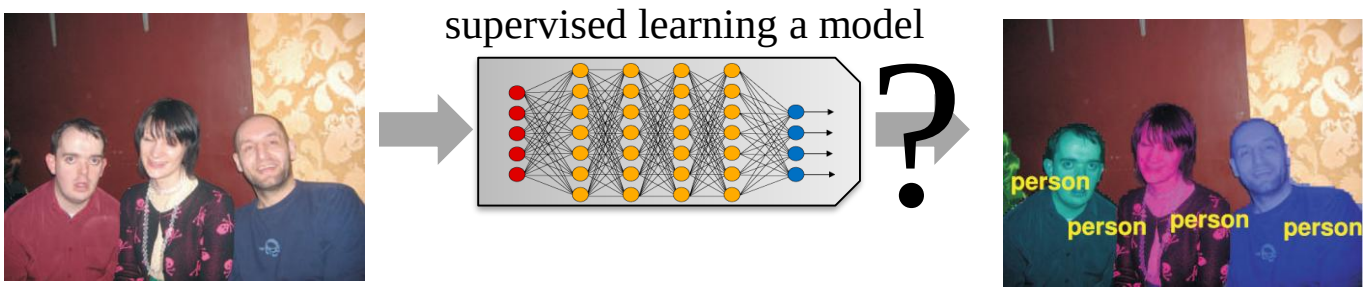



LeCun et al., "Backpropagation applied to handwritten zip code recognition", Neural computation, 1989
Krizhevsky et al., "Imagenet classification with deep convolutional neural networks", NeurIPS, 2012

# Difficulties in learning for instance segmentation

We can't do per-pixel classification

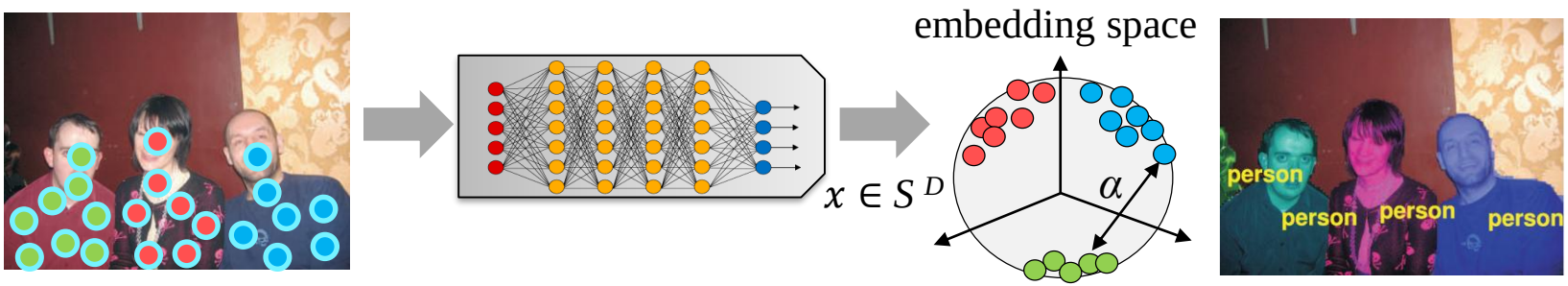
- varied number of objects
- permutable "labels"



# Pixel Embedding

**loss design**: for pixels $i$ and $j$, their feature embeddings' <u>cosine similarity</u> $s_{i\,j}$ should be:

$$\begin{cases} \rightarrow 1 & \text{if } i \text{ and } j \text{ are from the same object} \\ < \alpha & \text{otherwise} \end{cases}$$

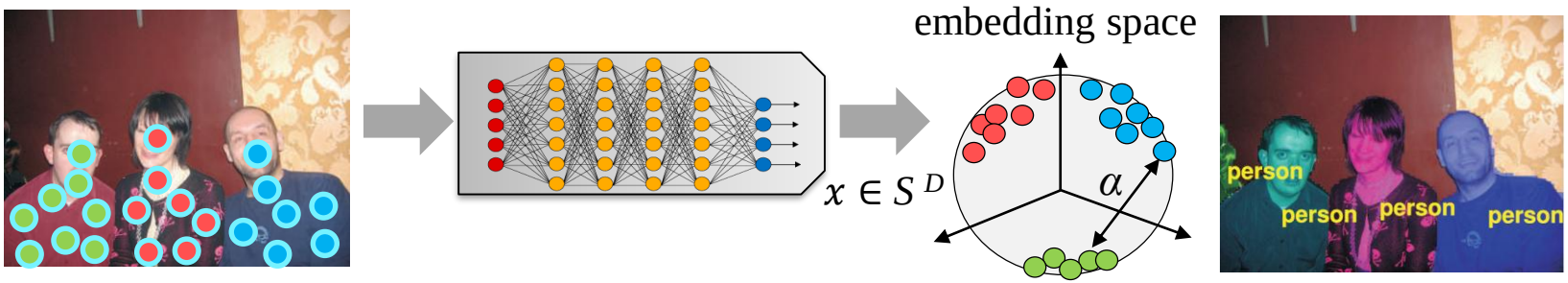



Kong & Fowlkes, "Recurrent Pixel Embedding for Instance Grouping", CVPR, 2018

# Pixel Embedding

**loss design**: for pixels *i* and *j*, their feature embeddings' <u>cosine similarity</u> $s_{ij} = \dfrac{x_i \cdot x_j}{||x_i||_2 \cdot ||x_j||_2}$ should be:

$$\begin{cases} \rightarrow 1 & \text{if } i \text{ and } j \text{ are from the same object} \longrightarrow (1 - s_{ij}) \\ < \alpha & \text{otherwise} \longrightarrow [s_{ij} - \alpha]_+ \end{cases}$$

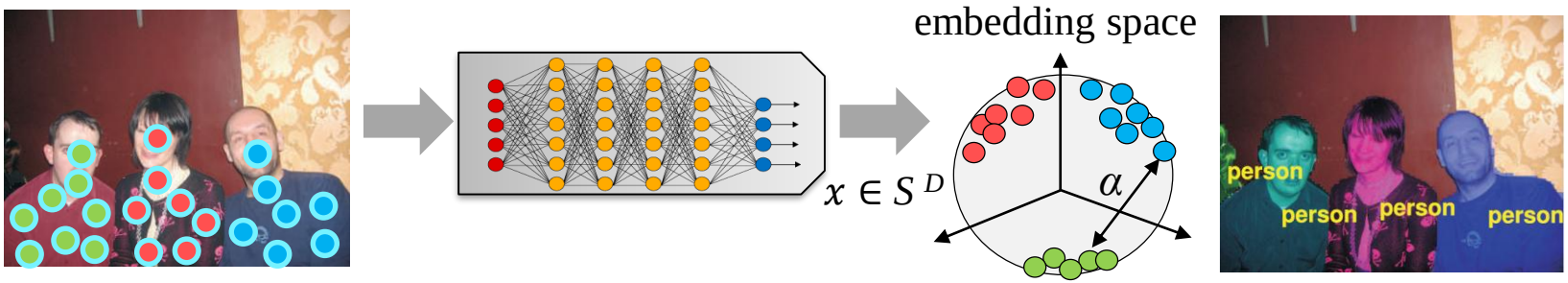



Kong & Fowlkes, "Recurrent Pixel Embedding for Instance Grouping", CVPR, 2018

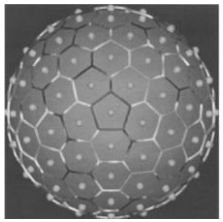
# Pixel Embedding

**loss design**: for pixels *i* and *j*, their feature embeddings' <u>cosine similarity</u> $s_{ij} = \dfrac{x_i \cdot x_j}{||x_i||_2 \cdot ||x_j||_2}$ should be:

$$\begin{cases} \rightarrow 1 & \text{if } i \text{ and } j \text{ are from the same object} \longrightarrow (1 - s_{ij}) \\ < \alpha & \text{otherwise} \longrightarrow [s_{ij} - \alpha]_+ \end{cases}$$




cosine similarity ⟷ spherical embedding



Saff & Kuijlaars. "Distributing many points on a sphere". The mathematical intelligencer, 1997.

# Distributing Many Points on a Sphere

E. B. SAFF AND A. B. J. KUIJLAARS

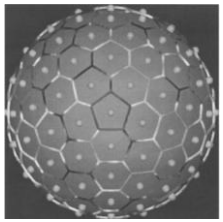
*The problem of distributing a large number of points* uniformly *over the surface of a sphere has not only inspired mathematical researchers, it has attracted the attention of biologists, chemists, and physicists working in such fields as viral morphology, crystallography, molecular structure, and electrostatics. In two dimensions, the anal-*

ogous problem is simply that of uniformly distributing points on the circumference of a disk, and equally spaced points provide an obvious answer. So we are faced with this question: What sets of points on the sphere imitate the role of the roots of unity on the unit circle?

One way such points can be generated is via optimization with respect to a suitable criterion such as "generalized energy." Although there is a large and growing literature concerning such optimal spherical configurations of $N$ points when $N$ is "small," here we shall focus on this question from an asymptotic perspective ($N \to \infty$).

Coulomb potential law is a challenging problem, sometimes referred to as the dual problem for stable molecules.

Certainly, uniformly distributing many points on the sphere has important applications to the field of computation. Indeed, quadrature formulas rely on appropriately chosen sampled data-points in order to approximate area integrals by taking averages in these points. Another example arises in the study of computational complexity, where M. Shub and S. Smale [22] encountered the problem of determining spherical points that maximize the product of their mutual distances.

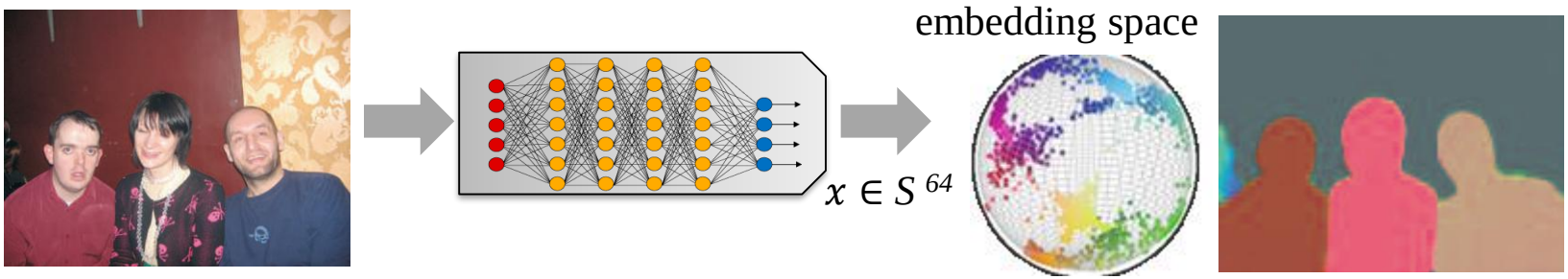


Saff & Kuijlaars. "Distributing many points on a sphere". The mathematical intelligencer, 1997.

# Pixel Embedding

**loss design**: for pixels *i* and *j*, their feature embeddings' <u>cosine similarity</u> $s_{ij} = \dfrac{x_i \cdot x_j}{||x_i||_2 \cdot ||x_j||_2}$ should be:

$$\begin{cases} \rightarrow 1 & \text{if } i \text{ and } j \text{ are from the same object} \longrightarrow (1 - s_{ij}) \\ < \alpha & \text{otherwise} \longrightarrow [s_{ij} - \alpha]_+ \end{cases}$$



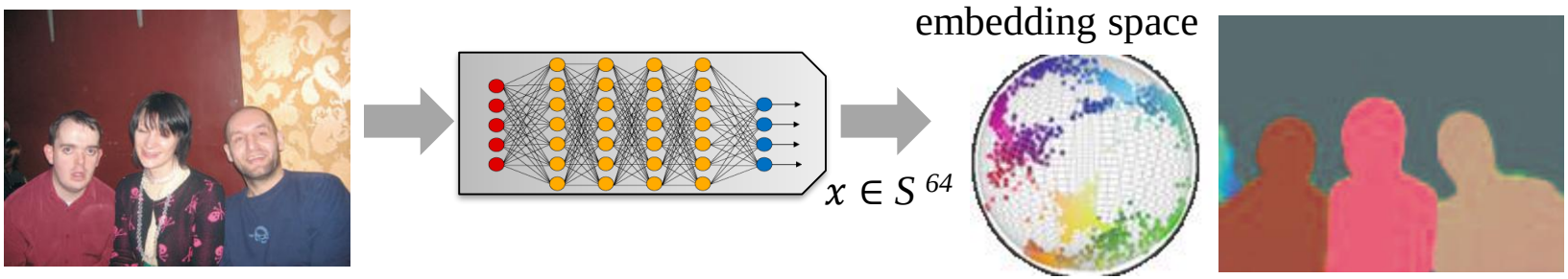

3-dim random projection from 64-dim embedding

Kong & Fowlkes, "Recurrent Pixel Embedding for Instance Grouping", CVPR, 2018

# Pixel Embedding

**loss design**: for pixels *i* and *j*, their feature embeddings' <u>cosine similarity</u> $s_{ij} = \dfrac{x_i \cdot x_j}{||x_i||_2 \cdot ||x_j||_2}$ should be:

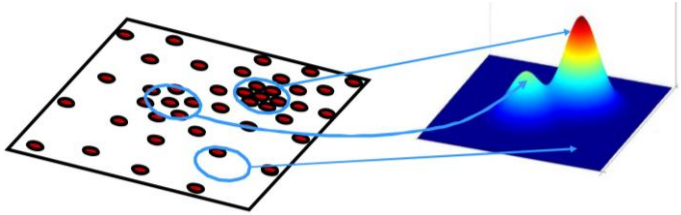
$$\begin{cases} \rightarrow 1 & \text{if } i \text{ and } j \text{ are from the same object} \longrightarrow (1 - s_{ij}) \\ < \alpha & \text{otherwise} \longrightarrow [s_{ij} - \alpha]_+ \end{cases}$$




3-dim random projection from 64-dim embedding

Mean Shift: iteratively seeking mode / maxima of a density function

Window size is correlated to the margin $\alpha$ in the loss function.

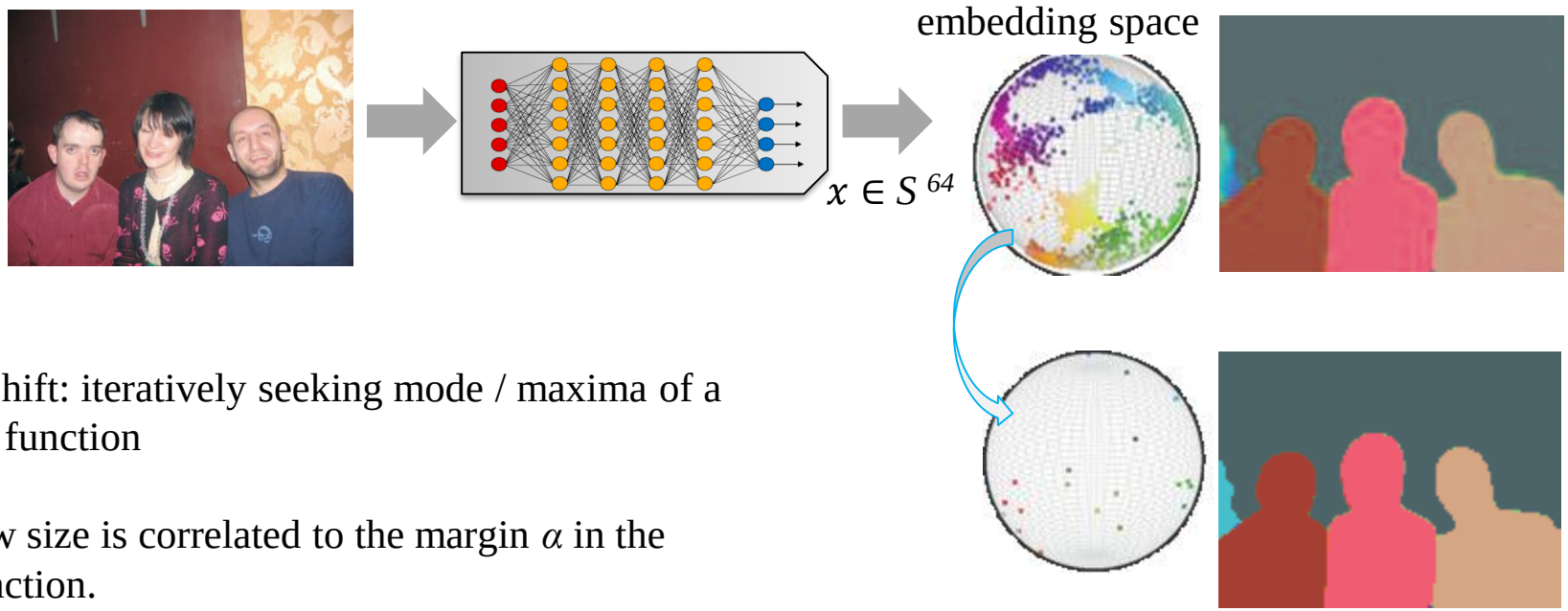



Fukunaga & Hostetler, "The Estimation of the Gradient of a Density Function, with Applications in Pattern Recognition", PAMI, 1975
Kobayashi & Otsu, "Von Mises-Fisher Mean Shift for Clustering on a Hypersphere", ICPR, 2010

# Pixel Grouping

**loss design**: for pixels *i* and *j*, their feature embeddings' <u>cosine similarity</u> $s_{ij} = \dfrac{x_i \cdot x_j}{||x_i||_2 \cdot ||x_j||_2}$ should be:

$$\begin{cases} \rightarrow 1 & \text{if } i \text{ and } j \text{ are from the same object} \longrightarrow (1 - s_{ij}) \\ < \alpha & \text{otherwise} \longrightarrow [s_{ij} - \alpha]_+ \end{cases}$$




Mean Shift: iteratively seeking mode / maxima of a density function

Window size is correlated to the margin $\alpha$ in the loss function.

Fukunaga & Hostetler, "The Estimation of the Gradient of a Density Function, with Applications in Pattern Recognition", PAMI, 1975
Kobayashi & Otsu, "Von Mises-Fisher Mean Shift for Clustering on a Hypersphere", ICPR, 2010

# Pixel Grouping

**loss design**: for pixels *i* and *j*, their feature embeddings' cosine similarity $s_{ij} = \dfrac{x_i \cdot x_j}{||x_i||_2 \cdot ||x_j||_2}$ should be:

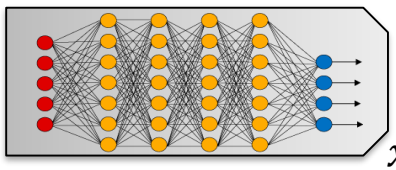
$$\begin{cases} \rightarrow 1 & \text{if } i \text{ and } j \text{ are from the same object} \longrightarrow (1 - s_{ij}) \\ < \alpha & \text{otherwise} \longrightarrow [s_{ij} - \alpha]_+ \end{cases}$$

$$\mathbf{S} = \mathbf{X}^T\mathbf{X}$$

$$\mathbf{G} = \exp(\delta^2(\mathbf{S} - 1)), \quad \mathbf{d} = \mathbf{G}^T\mathbf{1}, \quad \mathbf{q} = \mathbf{d}^{-1}$$

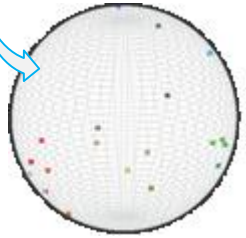
$$\mathbf{P} = \mathbf{G}\mathrm{diag}(\mathbf{q}), \text{ or } \mathbf{P} = (1-\eta)\mathbf{I} + \eta\mathbf{G}\mathrm{diag}(\mathbf{q})$$

$$\mathbf{Y} = \mathbf{X}\mathbf{P}$$



$x \in S^{64}$

embedding space



Mean Shift: iteratively seeking mode / maxima of a density function

Window size is correlated to the margin $\alpha$ in the loss function.

Miguel A. Carreira-Perpinán, "Fast Nonparametric Clustering with Gaussian Blurring Mean-Shift", ICML, 2006

# Von Mises Fisher Mean Shift

## L2 normalization in the loop

**Algorithm 1** : vMF Mean Shift

**[Normalization]** $||\boldsymbol{x}_i|| = 1, \forall i.$

**for** $i = 1$ **to** $N$ **do**

**[Init]** Start from the $i$-th sample: $\boldsymbol{y}_i^{(0)} = \boldsymbol{x}_i$.

**[Update]** Update the target points (vectors) until convergence by the following formula:

$$\boldsymbol{y}_i^{(t+1)} \leftarrow \frac{\sum_j^N \boldsymbol{x}_j g(\boldsymbol{x}_j' \boldsymbol{y}_i^{(t)}; \kappa)}{||\sum_j^N \boldsymbol{x}_j g(\boldsymbol{x}_j' \boldsymbol{y}_i^{(t)}; \kappa)||}. \quad (6)$$

**end for**

**[Postprocessing]** Merge close convergent points $(\boldsymbol{y}_i^{(\infty)'} \boldsymbol{y}_j^{(\infty)} > 1 - \epsilon)$ into the same cluster.

It's guaranteed to converge without gradient vanishing or exploding.

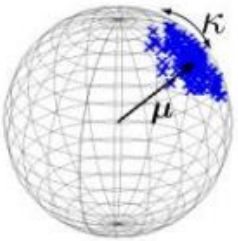
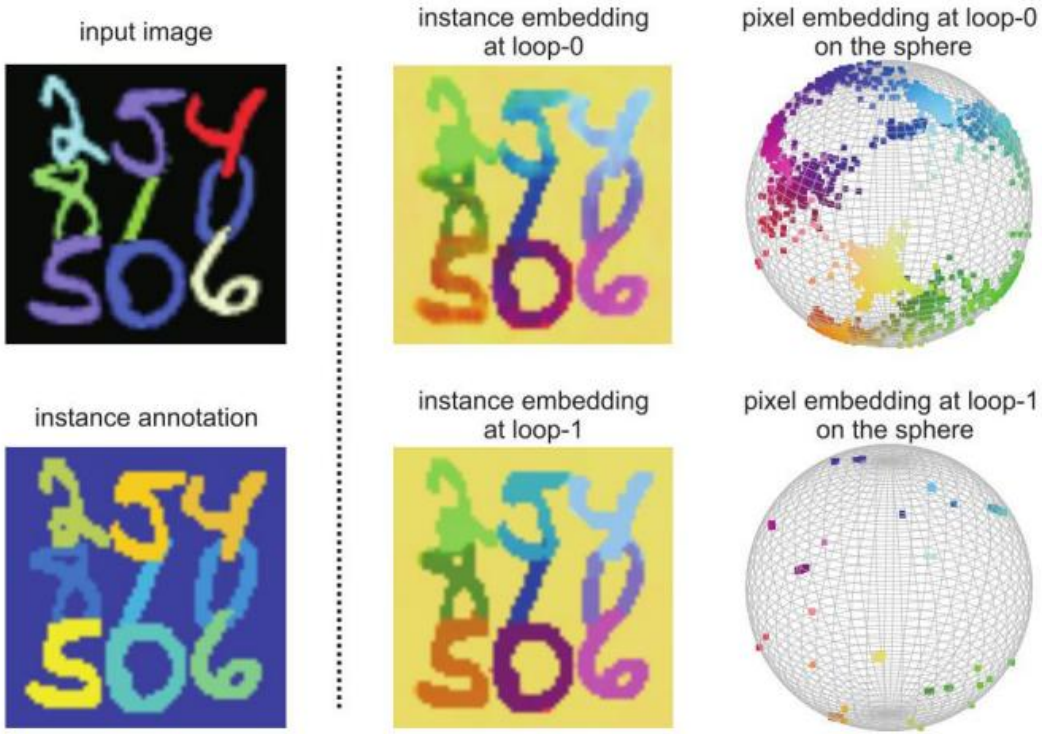



Figure 1. von Mises-Fisher distribution on a three-dimensional sphere.

# Learning with Pixel Grouping

running the von-Mises Fisher mean shift offline




Demonstration. After 10 iterations of mean-shift

Kong & Fowlkes, "Recurrent Pixel Embedding for Instance Grouping", CVPR, 2018

# Learning with Pixel Grouping

$$\mathbf{S} = \mathbf{X}^T\mathbf{X}$$

$$\mathbf{G} = \exp(\delta^2(\mathbf{S} - 1)), \quad \mathbf{d} = \mathbf{G}^T\mathbf{1}, \quad \mathbf{q} = \mathbf{d}^{-1}$$

$$\mathbf{P} = \mathbf{G}\mathrm{diag}(\mathbf{q}), \text{ or } \mathbf{P} = (1-\eta)\mathbf{I} + \eta\mathbf{G}\mathrm{diag}(\mathbf{q})$$

$$\mathbf{Y} = \mathbf{X}\mathbf{P}$$

Kong & Fowlkes, “Recurrent Pixel Embedding for Instance Grouping”, CVPR, 2018

# Learning with Pixel Grouping

$$\mathbf{S} = \mathbf{X}^T\mathbf{X}$$

$$\mathbf{G} = \exp(\delta^2(\mathbf{S}-1)), \quad \mathbf{d} = \mathbf{G}^T\mathbf{1}, \quad \mathbf{q} = \mathbf{d}^{-1}$$

$$\mathbf{P} = \mathbf{G}\mathrm{diag}(\mathbf{q}), \text{ or } \mathbf{P} = (1-\eta)\mathbf{I} + \eta\mathbf{G}\mathrm{diag}(\mathbf{q})$$

$$\mathbf{Y} = \mathbf{X}\mathbf{P}$$

mean shift grouping in the loop

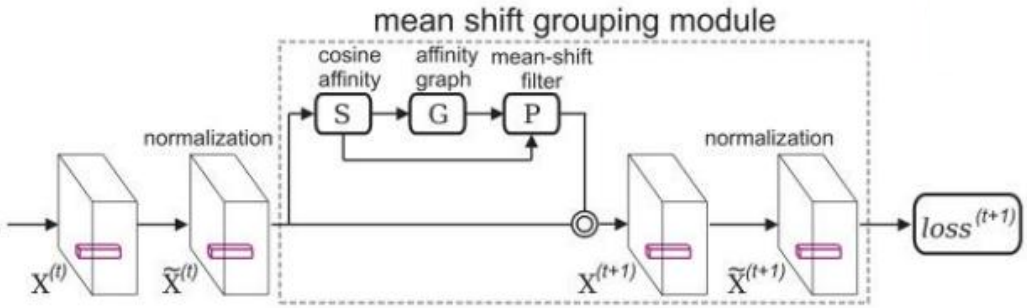
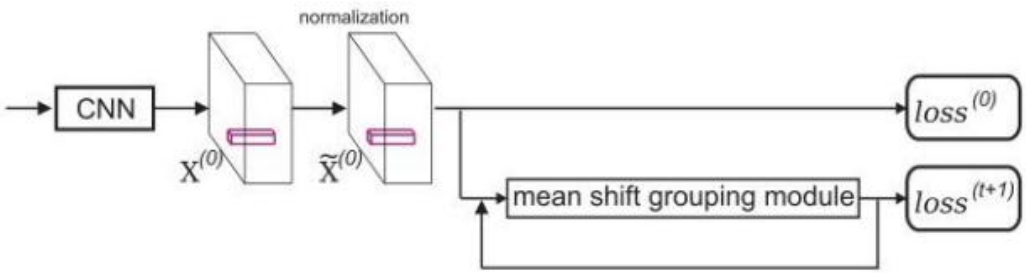



Figure 2: Mean shift grouping module.




Kong & Fowlkes, "Recurrent Pixel Embedding for Instance Grouping", CVPR, 2018

# Learning with Pixel Grouping

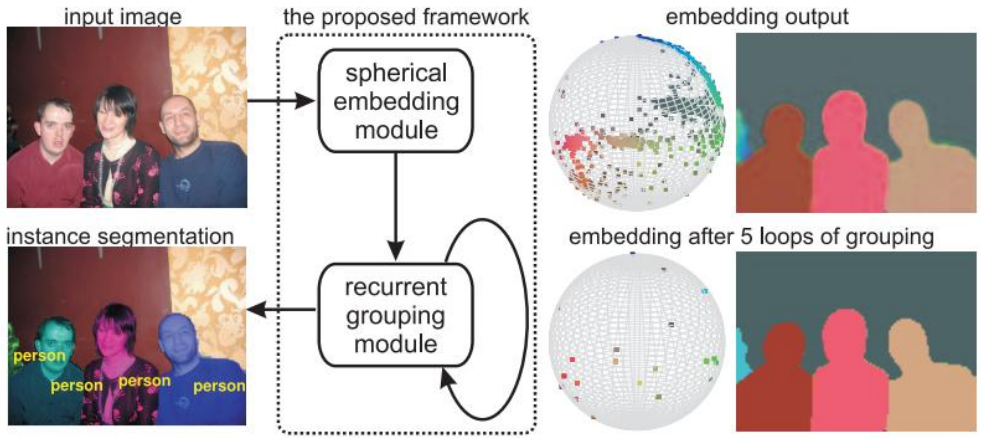
$$\mathbf{S} = \mathbf{X}^T\mathbf{X}$$

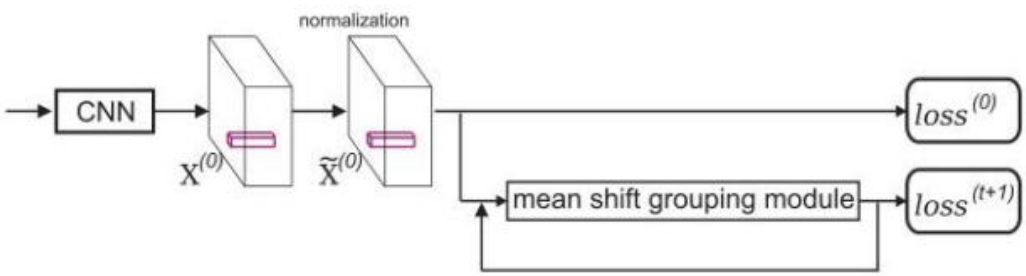
$$\mathbf{G} = \exp(\delta^2(\mathbf{S} - 1)), \quad \mathbf{d} = \mathbf{G}^T\mathbf{1}, \quad \mathbf{q} = \mathbf{d}^{-1}$$

$$\mathbf{P} = \mathbf{G}\mathrm{diag}(\mathbf{q}), \text{ or } \mathbf{P} = (1-\eta)\mathbf{I} + \eta\mathbf{G}\mathrm{diag}(\mathbf{q})$$

$$\mathbf{Y} = \mathbf{X}\mathbf{P}$$

mean shift grouping in the loop

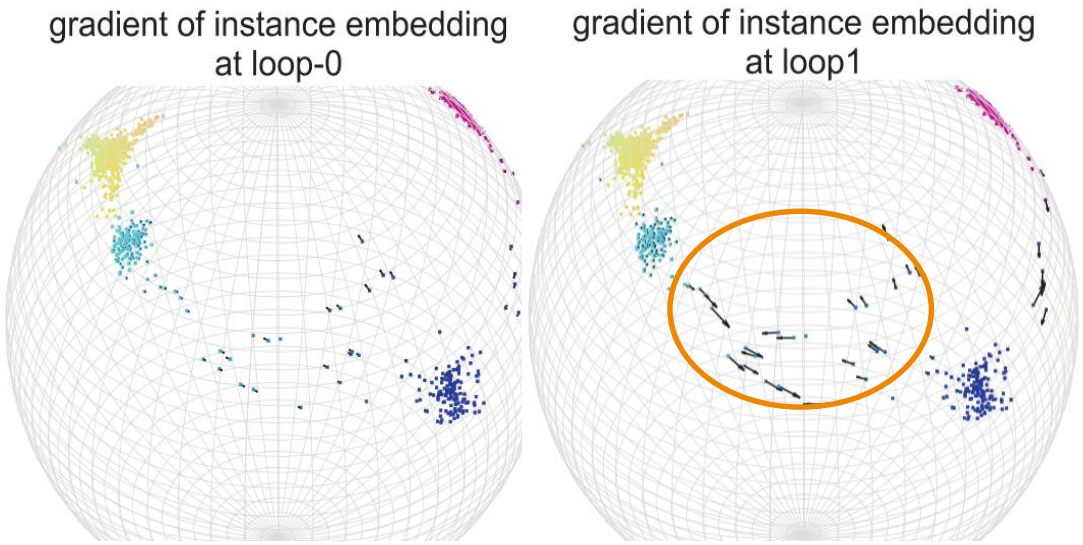


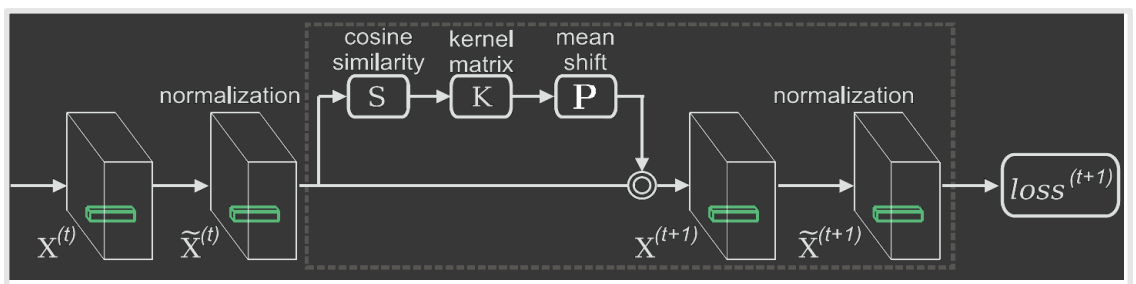



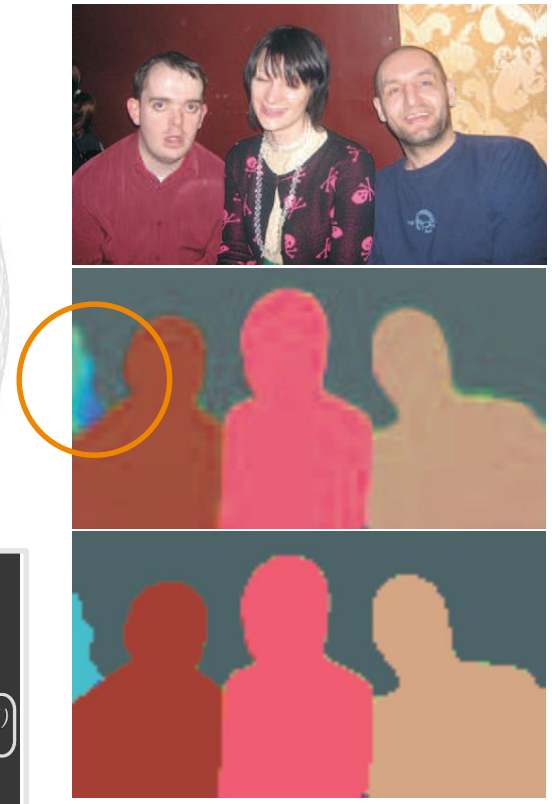

Kong & Fowlkes, "Recurrent Pixel Embedding for Instance Grouping", CVPR, 2018

# Learning with Pixel Grouping

Pixel grouping corrects embedding errors that won't be resolved in the subsequent clustering stage.



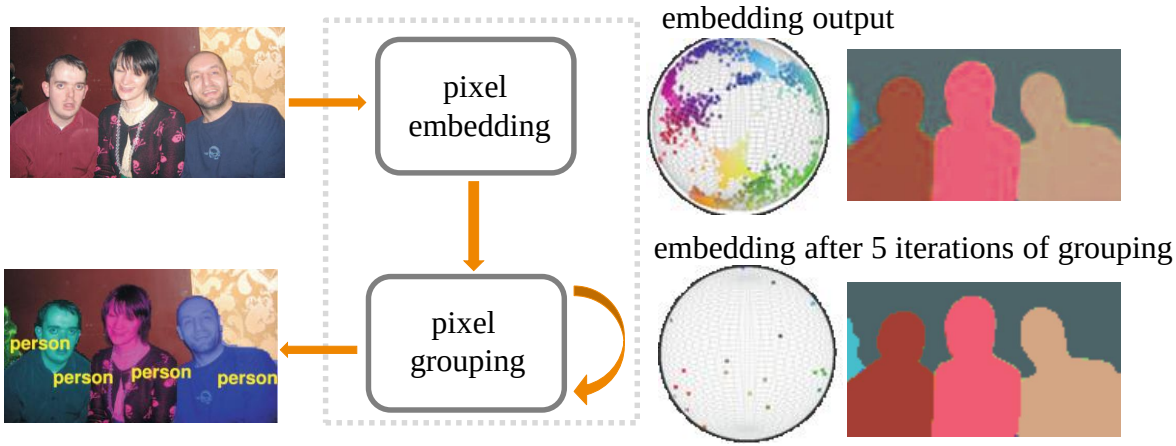



Kong & Fowlkes, “Recurrent Pixel Embedding for Instance Grouping”, CVPR, 2018

# Pixel Grouping

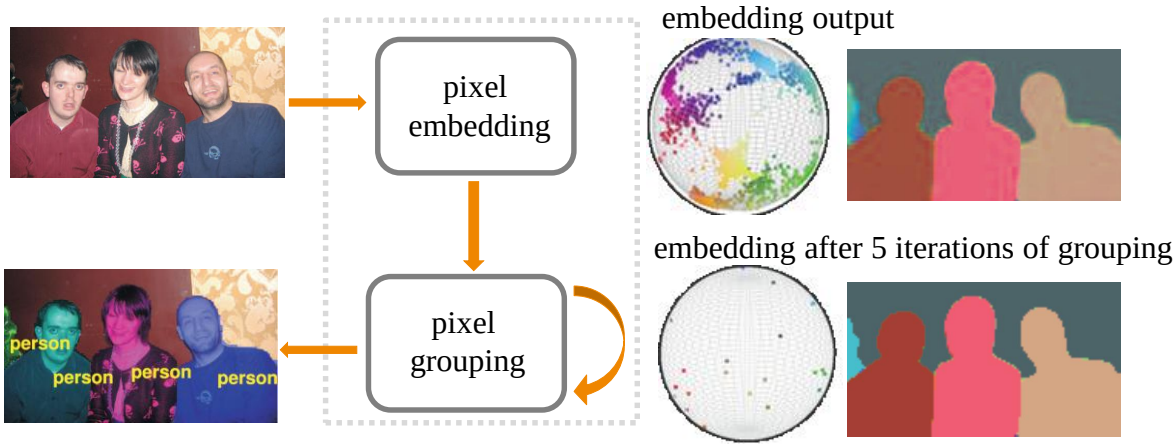
Ours: pixel embedding and grouping



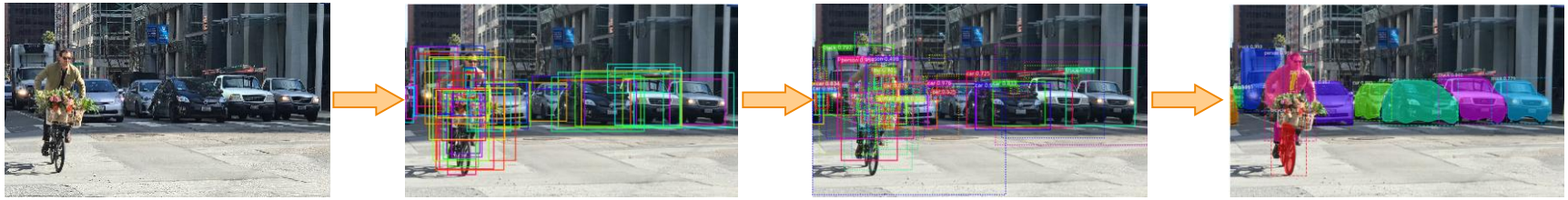

Kong & Fowlkes, “Recurrent Pixel Embedding for Instance Grouping”, CVPR, 2018

# Pixel Grouping vs. Prior Art

Ours: pixel embedding and grouping




Others, e.g., Mask R-CNN

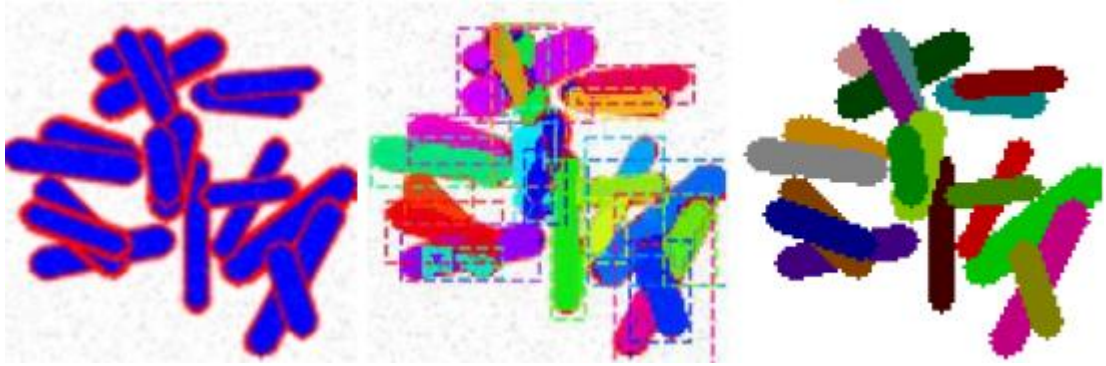


He et al. "Mask R-CNN." ICCV, 2017

# Pixel Grouping vs. Prior Art

Pixel Grouping significantly outperforms Mask R-CNN for overlapping objects.

Object instances | Mask R-CNN | Pixel Grouping



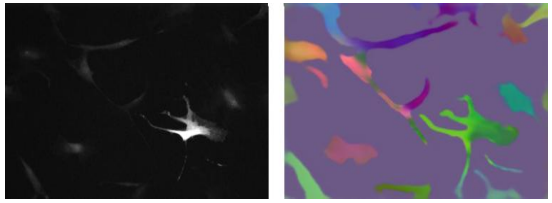
Sofiiuk et al. ICCV 2019

He et al. "Mask R-CNN." ICCV, 2017

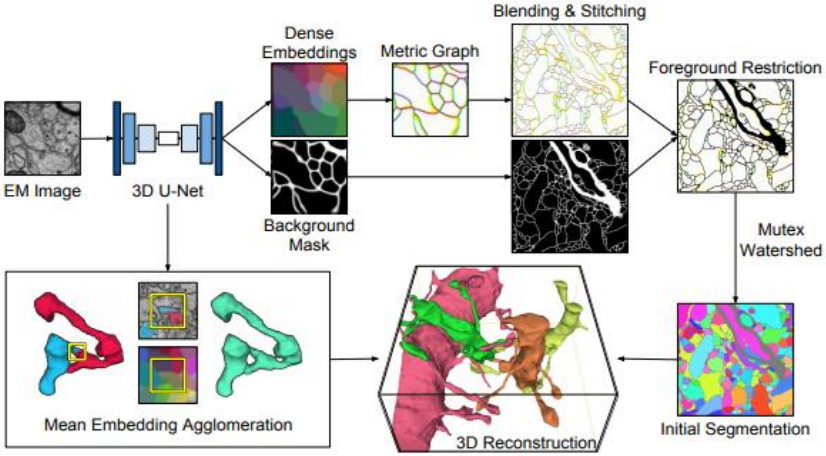
# Pixel Grouping for More Applications

Kulikov & Lempisky CVPR 2020

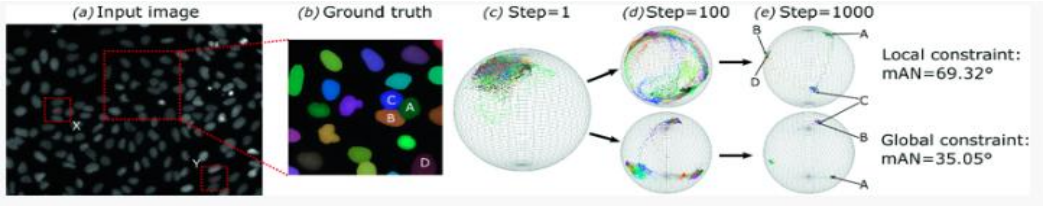
Payer et al. Medical Image Analysis 2019



Lee et al. IEEE Trans. Medical Imaging, 2021



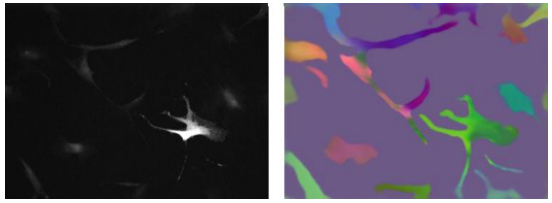

Chen et al. MICCAI 2019



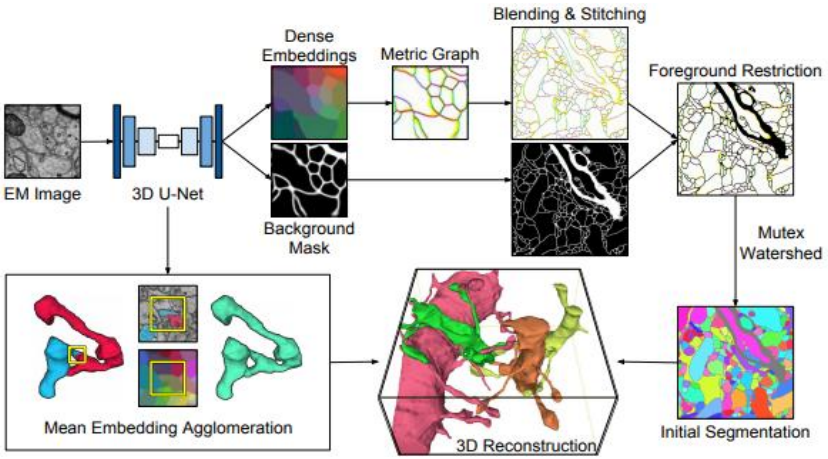
# Pixel Grouping for More Applications

Kulikov & Lempisky CVPR 2020

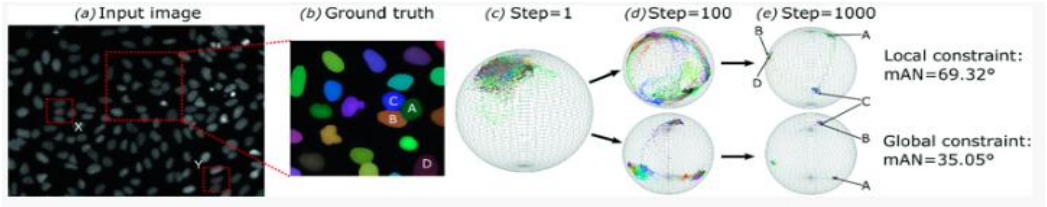
Payer et al. Medical Image Analysis 2019



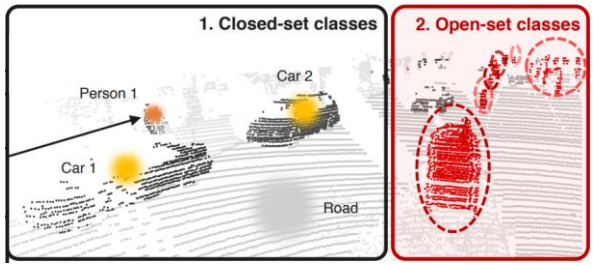
Lee et al. IEEE Trans. Medical Imaging, 2021



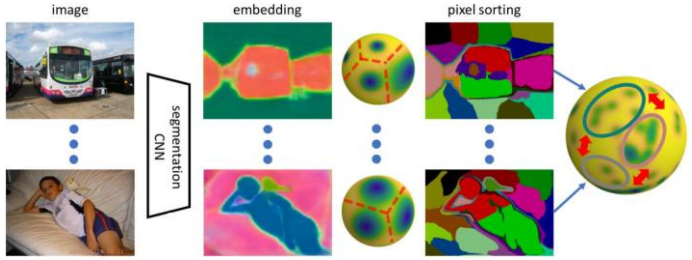

Chen et al. MICCAI 2019

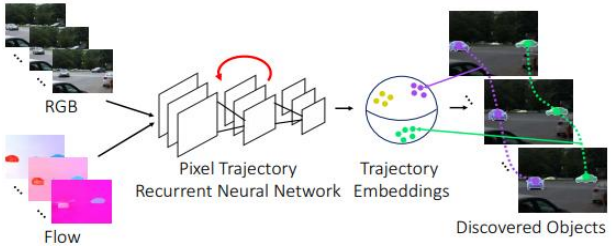



Wong et al. CoRL 2019




Hwang et al. ICCV 2019




Xie et al. CVPR 2019